

LoRA Fine-Tuning

Parameter-Efficient Model Adaptation — CS290W LLM09

LU Yuzhou

Today's Topics:

- 1 **LoRA Theory:** Low-rank matrix decomposition for weight updates
- 2 **LoRA Implementation:** Building LoRA layers from scratch
- 3 **PEFT Library:** Applying LoRA to real LLMs (Qwen2.5-0.5B)
- 4 **Supervised Fine-Tuning:** Training with LoRA on instruction data
- 5 **QLoRA:** 4-bit NF4 quantization + LoRA for consumer GPUs
- 6 **Variants & Production:** DoRA, rsLoRA, adapter save/load/merge

Why LoRA Matters

Full fine-tuning of a 7B model requires 28 GB (FP32) for weights alone. LoRA reduces trainable parameters by 100–1000 $\times$, enabling adaptation on consumer hardware.

- **Recommended Hardware:** AMD Ryzen AI Halo Processors
- **Environment:** ROCm + PyTorch via AUP Learning Cloud

The Fine-Tuning Problem

Full fine-tuning updates *all* model parameters for a downstream task.

Challenges

- **Memory:** Storing optimizer states + gradients for every parameter
- **Storage:** Each task needs a full model copy (7B params $\approx$ 14 GB in FP16)
- **Catastrophic Forgetting:** Full updates may overwrite pre-trained knowledge

Key Insight (Aghajanyan et al., 2020):

*The adaptation of pre-trained models to downstream tasks lies in a **low-dimensional subspace**.
We don't need to update all parameters.*

Solution: Learn a low-rank *incremental* update ΔW instead of the full W .

LoRA: Core Idea

Low-Rank Adaptation (Hu et al., 2021) decomposes the weight update matrix:

Traditional Fine-Tuning

$$W_{\text{new}} = W_{\text{original}} + \Delta W$$

Trainable parameters: $d \times k$ (full matrix).

LoRA Approach

$$\Delta W = A \cdot B^{\top}, \quad A \in \mathbb{R}^{d \times r}, \quad B \in \mathbb{R}^{k \times r}$$

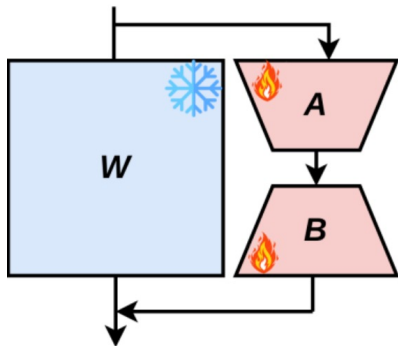
Trainable parameters: $r \times (d + k)$, where $r \ll \min(d, k)$.

Parameter Reduction:

$$\frac{d \times k}{r \times (d + k)} \quad (\text{typically } 100\text{--}1000\times)$$

Example: 4096×4096 matrix: 16M params. LoRA with $r = 16$: 131K params (122 $\times$ reduction).

Full Fine-Tuning vs LoRA

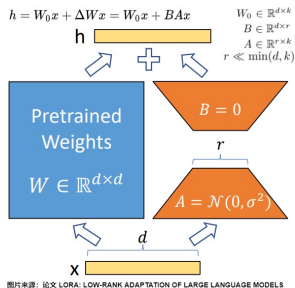


图片来源: 论文 Propulsion: Steering LLM with Tiny Fine-Tuning

- **Left (Full FT):** Update entire $d \times d$ weight matrix
- **Right (LoRA):** Freeze W_0 , learn low-rank $\Delta W = BA$ where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times d}$
- Parameters: $d^2 \rightarrow 2rd$ (e.g., $4096^2 \rightarrow 2 \times 16 \times 4096 = 131K$)

Image credit: "Exploring Transformer (22)—LoRA" [1]

LoRA Training & Inference



Training: Freeze W_0 , only update A (Gaussian init) and B (zero init).

$$W_{\text{new}} = W_0 + \frac{\alpha}{r} \cdot BA$$

Inference: Merge $W' = W_0 + \frac{\alpha}{r} BA$ — zero overhead, same architecture.

Image credit: “Exploring Transformer (22)—LoRA” [1]

Efficient Computation

Naïve approach: Compute $\Delta W = AB^\top$ ($d \times k$), then $\Delta W \cdot x$.

Efficient approach: Reorder matrix multiplications:

$$\underbrace{(xA)}_{\text{batch} \times r} \cdot B^\top \quad \text{instead of} \quad \underbrace{(AB^\top)}_{d \times k} \cdot x$$

Method	FLOPs	Intermediate Size
Naïve ($AB^\top$ then $\cdot x$)	$dk + dk \cdot n$	$d \times k$
Efficient (xA then $\cdot B^\top$)	$dr + kr \cdot n$	$n \times r$

For $d = k = 4096$, $r = 16$, $n = 1$: efficient is $\sim 128\times$ less compute.

Memory: No need to materialize full ΔW matrix.

Implementation: LoRALayer

```
class LoRALayer(nn.Module):
    def __init__(self, in_dim, out_dim, rank, alpha=1.0, dropout=0.0):
        super().__init__()
        self.A = nn.Parameter(torch.randn(in_dim, rank) / math.sqrt(rank))
        self.B = nn.Parameter(torch.zeros(rank, out_dim))
        self.scaling = alpha / rank
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        x = self.dropout(x)
        return (x @ self.A @ self.B) * self.scaling
```

Parameter count: $r \times (d_{\text{in}} + d_{\text{out}})$

Compression examples (512 $\rightarrow$ 256 layer):

- $r = 4$: 3,072 params vs 131,072 original ($42.7\times$ reduction)
- $r = 16$: 12,288 params ($10.7\times$ reduction)
- $r = 64$: 49,152 params ($2.7\times$ reduction)

Integration with Existing Layers

Wrapper pattern: Freeze original weights, add LoRA branch.

```
class LinearWithLoRA(nn.Module):
    def __init__(self, linear_layer, rank, alpha=1.0):
        super().__init__()
        self.linear = linear_layer
        self.lora = LoRALayer(linear_layer.in_features,
                               linear_layer.out_features, rank, alpha)

        for param in self.linear.parameters():
            param.requires_grad = False

    def forward(self, x):
        return self.linear(x) + self.lora(x)
```

Design Principles:

- **Frozen Base:** Original weights never change
- **Additive:** LoRA output added to original output
- **Selective:** Apply only to chosen layers
- **Composable:** Multiple adapters can be swapped

LoRA Application Strategies

Where to apply LoRA in a network?

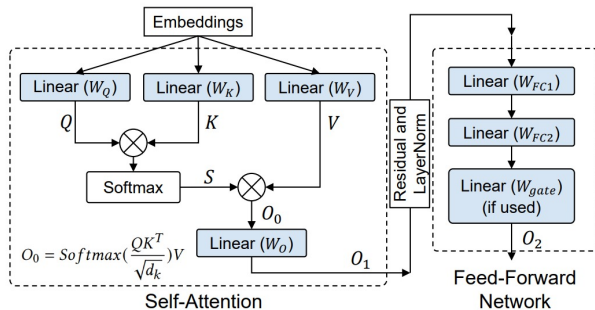
Strategy	Description
All Layers	Apply LoRA to every linear layer
Output Only	Only adapt the final classification layer
Input/Output	Adapt first and last layers
Attention Only	Focus on Q, K, V, O projections
Selective	Different ranks per layer based on importance

Recommended for LLMs: Target attention projections (Q, K, V, O) with optional MLP layers.

Layer-specific ranks:

- Larger / more important layers → higher rank
- Smaller / less critical layers → lower rank

Where to Place LoRA in Transformer?



图片来源: 论文 Fast and Efficient 2-bit LLM Inference on GPU: 2/4/16-bit in a Weight Matrix with Asynchronous Dequantization

Dense layers in Transformer:

- **Attention:** W^Q, W^K, W^V, W^O
- **FFN (MLP):** gate_proj, up_proj, down_proj

Original LoRA paper: Only W^Q and W^V

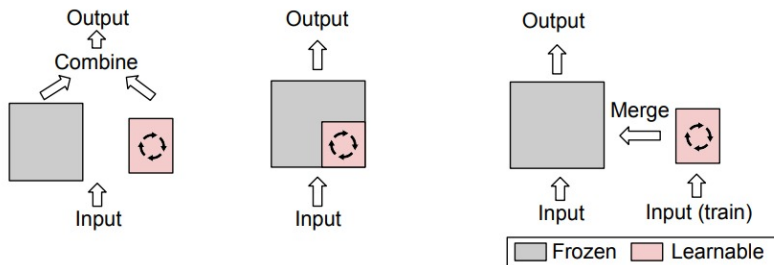
Best practice: All 4 attention projections (+ optionally FFN)

Blue boxes = dense layers suitable for LoRA

Image credit: "Exploring Transformer (22)—LoRA" [1]

PEFT Methods Taxonomy

(a) Additive PEFT (b) Selective PEFT (c) Reparameterization PEFT



图片来源：论文 Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey

LoRA belongs to “Reparameterization PEFT” — builds low-dimensional reparameterization of original weights.

Other categories: Additive (adapters, prompts), Selective (freeze subsets), Hybrid (combine methods).

Image credit: “Exploring Transformer (22)—LoRA” [1]

PEFT (Parameter-Efficient Fine-Tuning) is the industry standard for LoRA on LLMs.

One-Line LoRA:

```
from peft import LoraConfig, get_peft_model
config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=16, lora_alpha=32, lora_dropout=0.05,
    target_modules=['q_proj', 'k_proj', 'v_proj', 'o_proj'],
)
peft_model = get_peft_model(base_model, config)
```

Key configuration:

- `r`: Rank (typically 8–64)
- `lora_alpha`: Scaling factor (typically $2r$)
- `target_modules`: Which layers get LoRA adapters
- `task_type`: `CAUSAL_LM`, `SEQ_CLS`, etc.

LoRA on Qwen2.5-0.5B

Base model: Qwen2.5-0.5B (Alibaba) — 24 layers, 896 hidden dim

Module	Shape	LoRA Target?
q_proj	896 $\rightarrow$ 896	✓
k_proj	896 $\rightarrow$ 128	✓
v_proj	896 $\rightarrow$ 128	✓
o_proj	896 $\rightarrow$ 896	✓
gate_proj	896 $\rightarrow$ 4864	optional
up_proj	896 $\rightarrow$ 4864	optional
down_proj	4864 $\rightarrow$ 896	optional

Result with $r = 16$ on attention projections:

- Total params: 494M
- Trainable params: 2.16M
- **229 $\times$ parameter reduction** (0.44% trainable)

Supervised Fine-Tuning with LoRA

Training workflow:

- 1 Load instruction dataset (e.g., medical QA)
- 2 Format into instruction-response pairs
- 3 Train **only LoRA parameters** (base weights frozen)
- 4 Monitor loss convergence

Training Configuration

- Optimizer: AdamW ($\text{lr}=2 \times 10^{-4}$, $\text{weight_decay}=0.01$)
- Gradient clipping: $\text{max_norm}=1.0$
- Only LoRA params receive gradients

Observed results (medical QA, 10 epochs):

- Initial loss: 2.71 $\rightarrow$ Final loss: 0.31 (88.5% reduction)
- Model learns task-specific patterns through LoRA adapters

LoRA Variants: DoRA and rsLoRA

Drop-in improvements over standard LoRA:

DoRA (Weight-Decomposed LoRA, 2024)

Decomposes weight into **magnitude** and **direction**:

$$W' = m \cdot \frac{W + AB^T}{\|W + AB^T\|}$$

- Learnable magnitude vector m . Better performance at low ranks
- Slightly more parameters (+ magnitude vector)

rsLoRA (Rank-Stabilized LoRA, 2024)

Scaling factor: $\alpha/\sqrt{r}$ instead of α/r

- More stable training across different rank values
- Higher ranks maintain stronger adaptation

A and B Matrix: Asymmetric Roles

Lemma 4.5 (Generalization bounds on adapting A and/or B). Consider the algorithms of Definition 4.4. Assume that $\ell^{\mathbf{W}, \mathbf{b}}(\Delta \mathbf{W}, \tilde{Z})$ is σ -sub-Gaussian² under $(\Delta \mathbf{W}, \tilde{Z}) \sim P_{\Delta \mathbf{W} | \mathbf{W}, \mathbf{b}} \times \mu$. Then,

$$\begin{aligned} |\text{gen}(\mu, \mathcal{A}_{BA})| &\leq \sqrt{\frac{2rq\sigma^2 \ln 2}{n} \sum_{i \in \mathcal{I}} (d_{in}^{(i)} + d_{out}^{(i)})}, \\ |\text{gen}(\mu, \mathcal{A}_B)| &\leq \sqrt{\frac{2rq\sigma^2 \ln 2}{n} \sum_{i \in \mathcal{I}} d_{out}^{(i)}}, \\ |\text{gen}(\mu, \mathcal{A}_A)| &\leq \sqrt{\frac{2rq\sigma^2 \ln 2}{n} \sum_{i \in \mathcal{I}} d_{in}^{(i)}}. \end{aligned}$$

图片来源: Asymmetry in Low-Rank Adapters of Foundation Models

A matrix: Feature extractor

- Captures cross-domain commonality
- Can be shared across tasks
- Less critical to update

B matrix: Task adapter

- Captures task-specific differences
- More important than A
- Updating B alone $\approx$ full LoRA quality

Image credit: “Exploring Transformer (22)—LoRA” [1]

Variant Comparison

Variant	Scaling	Extra Overhead	Best For
Standard LoRA	α/r	none	General purpose
rsLoRA	$\alpha/\sqrt{r}$	none	High rank scenarios
DoRA	α/r	magnitude vec.	Low rank scenarios

Qwen2.5-0.5B comparison ($r = 16$, $\alpha = 32$):

Variant	Trainable Params	Scaling Factor
Standard LoRA	2,162,688	$\alpha/r = 2.00$
rsLoRA	2,162,688	$\alpha/\sqrt{r} = 8.00$
DoRA	2,211,840	$\alpha/r = 2.00$

All variants are drop-in replacements — just change LoraConfig flags!

Production Workflow: Save, Load, Merge

Critical for deployment:

- 1 **Save:** Store only adapter weights (tiny: $\sim$ MB vs $\sim$ GB for full model)
- 2 **Load:** Reuse adapter on same or different base model
- 3 **Merge:** Combine $W' = W + \frac{\alpha}{r}AB^T$ into base model — zero inference overhead
- 4 **Multi-adapter:** Hot-swap task-specific adapters at runtime

Storage Example

- 100 task-specific adapters: $\sim$ 100 MB total
- 100 full fine-tuned models: $\sim$ 100 GB total
- **1000 $\times$ storage savings!**

```
model.save_pretrained('./my_adapter')  
loaded = PeftModel.from_pretrained(base, './my_adapter')  
merged = loaded.merge_and_unload()
```

Decision Guide: When to Use What?

Scenario	Method	Reason
Plenty of GPU memory	Full fine-tuning	Best quality
Limited GPU memory	LoRA ($r = 16\text{--}64$)	Good quality, low memory
Very limited memory	QLoRA (4-bit)	Minimum memory
Low-rank tasks	DoRA	Better at low r
High-rank tasks	rsLoRA	Stable scaling
Multi-task serving	LoRA + hot-swap	One base, many adapters
Edge deployment	Merge LoRA	Zero overhead

Typical starting point for LLMs:

- $r = 16$, $\alpha = 32$ (scaling = 2.0)
- Target: Q, K, V, O projections (+ optionally MLP layers)
- Dropout: 0.05
- Learning rate: 2×10^{-4} to 3×10^{-4}

Lab: What You'll Implement

Part 1: LoRA from Scratch

- Build `LoRALayer` with proper initialization ($A \sim \mathcal{N}$, $B = 0$)
- Build `LinearWithLoRA` wrapper with frozen base weights
- Test different rank/alpha configurations
- Apply to multi-layer network with various strategies

Part 2: PEFT on Real LLMs

- Load Qwen2.5-0.5B with transformers
- Apply LoRA via `get_peft_model()`
- Fine-tune on medical QA dataset
- Test QLoRA with 4-bit NF4 quantization

Part 3: Production Workflow

- Save/load adapters
- Merge LoRA into base model
- Compare DoRA vs rsLoRA vs standard LoRA

Summary: Key Takeaways

- ❶ **LoRA Core:** $\Delta W = AB^\top$ with $r \ll \min(d, k)$
 - 100–1000× parameter reduction
 - $B = 0$ initialization ensures stable start
- ❷ **Efficient Computation:** $(xA)B^\top$ avoids materializing full ΔW
- ❸ **Integration:** Freeze base weights, add LoRA branch, train only adapters
- ❹ **PEFT:** One-line LoRA for any HuggingFace model
- ❺ **QLoRA:** 4-bit NF4 + LoRA = fine-tune 7B on 6 GB VRAM
- ❻ **Variants:** DoRA (weight-decomposed), rsLoRA (rank-stabilized)
- ❼ **Production:** Save/load adapters (MB), merge for zero-overhead inference

- **LoRA Paper:** Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models” (2021)
arXiv:2106.09685
- **QLoRA Paper:** Dettmers et al., “QLoRA: Efficient Finetuning of Quantized LLMs” (2023)
arXiv:2305.14314
- **DoRA Paper:** Liu et al., “DoRA: Weight-Decomposed Low-Rank Adaptation” (2024)
arXiv:2402.09353
- **rsLoRA:** Kalajdzievski, “A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA” (2024)
arXiv:2312.03732
- **HuggingFace PEFT:** github.com/huggingface/peft
- **Blog Reference:** “Exploring Transformer (22)—LoRA” (RossiXYZ, 2025)
cnblogs.com/rossiXYZ/p/18805103
- **Lab Notebook:** [LLM09-LoRA-Fine-Tuning.ipynb](#)



RossiXYZ, “Exploring Transformer (22)—LoRA,” Cnblogs Blog, Apr. 2025.
cnblogs.com/rossiXYZ/p/18805103



E. J. Hu *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models,” *arXiv preprint arXiv:2106.09685*, 2021.



T. Dettmers *et al.*, “QLoRA: Efficient Finetuning of Quantized LLMs,” *arXiv preprint arXiv:2305.14314*, 2023.



S. Liu *et al.*, “DoRA: Weight-Decomposed Low-Rank Adaptation,” *arXiv preprint arXiv:2402.09353*, 2024.



D. Kalajdzievski, “A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA,” *arXiv preprint arXiv:2312.03732*, 2024.